AutoIA V1 Tutorial Manual

Applies to: AutoIA `1.0.1`
Purpose: learn AutoIA from the smallest zero-knob call through path-aware engineering code, implicit solves, fixed-data interpolation, repeated certification, and application-specific theorem extensions.

AutoIA is designed to certify ordinary engineering Python over a Cartesian box of uncertain physical inputs. The central user contract is simple:

```
result = autoia(model, box)
```

If the call returns normally and `result.machine_rigorous` is true, every reported output interval is intended to contain the exact real output of the supported program semantics for every physical input in the supplied box. If AutoIA cannot establish the required proof, it fails closed with a typed `AutoIAError` and returns no object that is presented as rigorous.

This tutorial is intentionally progressive. It explains what the numbered examples demonstrate, but it also gives copy-paste code that can be run independently.

---

1. Installation and first check

Install the wheel into the Python environment that will execute your engineering code:

```
python -m pip install --upgrade --force-reinstall autoia-1.0.1-py3-none-any.whl
```

Verify the installed version:

```
import autoia
print(autoia.__version__)
```

Expected:

```
1.0.1
```

The minimum imports for ordinary use are:

```
from autoia import autoia, Interval
```

SciPy is optional. AutoIA does not require SciPy merely to install or to certify generic NumPy/Python code. SciPy is needed when the user's model itself uses one of the supported SciPy semantics such as `root_scalar`, `fsolve`, or `RegularGridInterpolator`.

---

2. Your first rigorous enclosure

Consider

$$f(x) = (x+1)^2, \qquad x \in [-0.2, 0.3].$$

The corresponding Python is ordinary scalar code:

```
from autoia import autoia, Interval


def model(p):
    x = p[0]
    return x*x + 2.0*x + 1.0


result = autoia(model, [Interval(-0.2, 0.3)])

print("lower =", result.lower)
print("upper =", result.upper)
print("width =", result.width)
print("machine rigorous =", result.machine_rigorous)
print(result.explain_configuration())
```

The vector `p` contains the uncertain physical inputs. Here there is one input, so `p[0]` is the uncertain scalar `x`.

What AutoIA does automatically

For the zero-knob call, AutoIA uses a polynomial/order-4 baseline and a bounded automatic configuration policy. The automatic selector can retry a higher polynomial order when truncation/remainder dominates retained physical dependence, or try the shared-generator representation when approximation-error reuse suggests that downstream cancellation matters.

Each successful attempt is independently rigorous. If several attempts succeed, AutoIA intersects their output intervals componentwise. This means the selector affects cost and sharpness, not the soundness argument.

---

3. Multiple uncertain inputs

A Cartesian uncertainty box is written as one `Interval` per physical input:

```
from autoia import autoia, Interval


def model(p):
```

```
    pressure = p[0]
    area = p[1]
    return pressure * area


box = [
    Interval(9.5e6, 10.5e6),
    Interval(0.009, 0.011),
]

result = autoia(model, box)
print(result)
```

The declared input set is

$$P = [9.5 \times 10^6, 10.5 \times 10^6] \times [0.009, 0.011].$$

AutoIA V1 uses Cartesian input boxes. Correlated physical input sets such as ellipsoids or arbitrary polytopes are not first-class input domains in V1.

3.1 Choosing what your interval endpoints mean

AutoIA 1.0.1 makes the input-specification boundary explicit. There are three constructors with intentionally different meanings.

```
from autoia import Interval

# Original V1 behavior: these stored binary64 numbers are the exact endpoints.
I_float = Interval(0.1, 0.3)

# The intended endpoints are the exact decimals 1/10 and 3/10.
I_exact = Interval.enclosing("0.1", "0.3")

# Pad already stored floats by one adjacent binary64 value on each side.
I_pad = Interval.outward(0.1, 0.3)
```

This distinction matters because Python's stored binary64 `0.1` is slightly *larger* than the exact real number `1/10`, while stored binary64 `0.3` is slightly *smaller* than `3/10`. Therefore `Interval(0.1, 0.3)` does not exactly mean the decimal interval `[1/10,3/10]`.

Use `Interval.enclosing` when the physical endpoints themselves are intended as exact decimal or rational quantities. It accepts decimal strings, `decimal.Decimal`, `fractions.Fraction`, and integers:

```
from decimal import Decimal
from fractions import Fraction
```

```
A = Interval.enclosing("0.1", "0.3")
B = Interval.enclosing(Decimal("0.1"), Decimal("0.3"))
C = Interval.enclosing(Fraction(1, 10), Fraction(3, 10))
```

Ordinary floats are deliberately rejected by `Interval.enclosing`, because once `0.1` has become a float AutoIA cannot know whether the user meant the exact decimal `1/10` or the stored binary64 value itself.

`Interval.outward` is different. It is only a one-ulp conservative pad around already stored floats. It does not prove the provenance of values computed outside AutoIA.

The chosen input semantics are visible after certification:

```
result = autoia(model, [I_exact])
print(result.input_semantics)
print(result.input_specification)
print(result.physical_input_box)
```

3.2 What about constants throughout my engineering code?

You normally leave them alone. Code such as

```
def model(p):
    return 0.37*p[0] + 12.5*p[1] - 4.8
```

keeps the V1 rule that stored binary64 constants are interpreted as exact real constants equal to their stored values. AutoIA does not ask you to rewrite every floating literal.

If a coefficient is physically uncertain, expose it as another uncertain input. If a quantity is genuinely mathematical and exact, prefer an exact formulation when practical; for example `x/10` uses the exactly represented integer 10 and the supported real division semantics. Do not assume AutoIA silently turns an ordinary source literal `0.1` back into the mathematical rational `1/10`.

Example 26 is a runnable demonstration of all three interval constructors and the result provenance. See also `INPUT_SPECIFICATION.md` at the repository root.

---

4. Elementary nonlinear functions

Common NumPy and `math` functions are locally redirected to certified semantics when AutoIA values reach them.

```
import numpy as np
from autoia import autoia, Interval
```

```python
def model(p):
    x = p[0]
    return np.exp(x) + np.sin(x) + x*x


result = autoia(model, [Interval(-0.2, 0.3)])
print(result)
```

Important supported examples include:

```
np.sqrt(x)
np.exp(x)
np.log(x)
np.log10(x)
np.sin(x)
np.cos(x)
```

and the analogous `math` forms when visible in the prepared function's global namespace.

AutoIA does not prove these functions by calling NumPy and padding the answer. The trusted layer evaluates outward-rounded interval kernels and the enclosure layer composes those kernels with retained polynomial/generator dependence.

Domain-sensitive functions

For

```
np.sqrt(x)
```

AutoIA must prove that the physical computation is real-valued over the box. For example:

```python
import numpy as np
from autoia import autoia, Interval


def model(p):
    return np.sqrt(p[0])


result = autoia(model, [Interval(0.0, 4.0)])
print(result)
```

The exact boundary `x=0` is allowed.

If a broad internal enclosure makes a domain proof inconclusive, AutoIA can request physical-box refinement and re-execute on independently certified child boxes. This is proof-driven subdivision, not clipping.

---

5. Powers and reciprocals

Supported examples include integer powers and constant real powers on their supported real branches:

```
from autoia import autoia, Interval


def model(p):
    x = p[0]
    return [
        x**4,
        (2.0 + x)**1.25,
        (2.0 + x)**-0.5,
    ]


result = autoia(model, [Interval(-0.5, 0.5)])
print(result)
```

An uncertain exponent such as

```
x ** p[1]
```

is not a V1 semantic.

A reciprocal requires a proof that the denominator is separated from zero. If the available broad-box enclosure crosses zero but the model may still be valid, automatic mode can subdivide the physical box and retry before failing.

---

6. Vector outputs and NumPy arrays

AutoIA supports ordinary engineering assembly patterns. Within the private prepared execution namespace, common array constructors default to object arrays when needed so uncertain scalar values can be stored in them.

```
import numpy as np
from autoia import autoia, Interval

COEFF = np.array([1.5, -2.0, 0.25])


def model(p):
    x, y = p
    v = np.array([
```

```
        x + y,
        x*y,
        x*x + 0.5*y,
    ])
    weighted = np.dot(COEFF, v)
    return np.array([
        v[0],
        v[1],
        v[2],
        weighted,
        np.sum(v),
    ])


result = autoia(
    model,
    [Interval(0.9, 1.1), Interval(1.8, 2.2)],
)

print(result.lower_shaped)
print(result.upper_shaped)
```

Supported reduction/composition operations include important forms of `sum`, `prod`, `mean`, `dot`, `matmul`, `inner`, `outer`, and `trace`, plus supported norms.

---

7. Parameter-dependent linear systems

A central semantic primitive is

`np.linalg.solve(A, b)`

when `A` and/or `b` contain AutoIA-dependent entries.

Example: two springs in a small static system.

```
import numpy as np
from autoia import autoia, Interval


def spring_model(p):
    k1, k2, force = p
    A = np.array([
        [k1 + k2, -k2],
        [-k2,      k2],
    ])
    b = np.array([force, 0.0])
    return np.linalg.solve(A, b)
```

```
result = autoia(
    spring_model,
    [
        Interval(90.0, 110.0),
        Interval(900.0, 1100.0),
        Interval(9.5, 10.5),
    ],
)

print(result)
print("certificates =", len(result.certificates))
```

NumPy's binary64 solve is not the proof. AutoIA uses a nominal solve/preconditioner as proposal data and independently checks a residual/contraction theorem over the full matrix family.

This pattern is important throughout AutoIA: numerical libraries can propose; AutoIA must verify.

---

8. Uncertain Python control flow

AutoIA can certify ordinary scalar `if` statements whose truth depends on uncertainty.

```
from autoia import autoia, Interval


def model(p):
    x = p[0]
    if x >= 0.0:
        return x*x + x
    return x*x - 2.0*x


result = autoia(model, [Interval(-1.0, 1.0)])
print(result)
print("branches =", len(result.branch_results))
```

AutoIA does not immediately replace both branches by one independent interval and continue. Each feasible path is re-executed and carried through downstream calculations separately. The final reported interval is the hull of independently certified path results.

Exact closed endpoints

Strict comparison semantics matter. Suppose:

```
from autoia import autoia, Interval


def model(p):
    x = p[0]
    if x > 1.0:
        raise ValueError("impossible on this box")
    if x < 1.0:
        return x*x
    return 1.0


result = autoia(model, [Interval(0.95, 1.0)])
print(result)
```

Although outward-rounded polynomial ranges may differ from exact endpoints by a few ulps, AutoIA carries a proof-only exact affine shadow for simple affine comparisons. Therefore `x > 1.0` is rigorously recognized as impossible on the closed physical box.

---

9. Manual subdivision and adaptive subdivision

Subdivision is a proof/sharpness strategy. Each child is independently certified.

Prescribed subdivision

```
import numpy as np
from autoia import autoia_subdivide, Interval


def model(p):
    x = p[0]
    return np.exp(-8.0*x*x) + 0.2*x


result = autoia_subdivide(
    model,
    [Interval(-1.0, 1.0)],
    split_counts=[4],
)
print(result)
```

Adaptive subdivision

```
import numpy as np
from autoia import autoia_adaptive_subdivide, Interval, SubdivisionPolicy


def model(p):
    x = p[0]
    return np.exp(-8.0*x*x) + 0.2*x


result = autoia_adaptive_subdivide(
    model,
    [Interval(-1.0, 1.0)],
    subdivision_policy=SubdivisionPolicy(
        max_branches=8,
        max_depth=4,
    ),
)
print(result)
```

The split score is controller logic, not trusted proof arithmetic. A bad split can waste time; it cannot make a child certificate false.

---

10. Polynomial, interval, and generator representations

Most users should not choose a representation manually. The explicit controls exist for research, reproducibility, and diagnostics.

```
from autoia import autoia, Interval


def model(p):
    x = p[0]
    y = x*x
    return [y, y-y]


box = [Interval(-1.0, 1.0)]

poly = autoia(model, box, order=1, representation="polynomial")
gen = autoia(model, box, order=1, representation="generator")
interval = autoia(model, box, representation="interval")

print("polynomial:", poly)
```

```
print("generator:", gen)
print("interval:", interval)
```

Why generators exist

A polynomial enclosure has the conceptual form

$$z = P(q) + R.$$

A generator enclosure has the conceptual form

$$z = P(q) + \sum_j g_j \varepsilon_j + R, \qquad \varepsilon_j \in [-1, 1].$$

The `q` variables are the original centered physical uncertain inputs. The auxiliary `epsilon` variables retain correlation introduced by approximation error. If the same approximation is reused twice, the same generator identifier is reused; downstream subtraction can therefore cancel it.

---

11. Expert polynomial ranging

The retained polynomial still has to be ranged over the physical box. Expert controls expose several strategies:

```
from autoia import autoia, Interval


def model(p):
    x, y = p
    return x*x + 0.5*x*y + 0.1*y*y


box = [Interval(-0.8, 0.7), Interval(-0.5, 0.9)]

termwise = autoia(model, box, representation="polynomial", order=4,
                  range_method="termwise")
mono = autoia(model, box, representation="polynomial", order=4,
              range_method="monotonicity")
bern = autoia(model, box, representation="polynomial", order=4,
              range_method="bernstein")
```

`termwise` bounds each monomial independently. `monotonicity` uses derivative-sign proofs to fix monotone coordinates at extremizing endpoints. `bernstein` converts the polynomial exactly to Bernstein form over a unit box and bounds it

using the Bernstein coefficients. Automatic mode uses cost-aware combinations rather than blindly invoking Bernstein everywhere.

---

12. Normal distributions and error functions

AutoIA exports certified normal-function helpers:

```
from autoia import (
    autoia,
    Interval,
    standard_normal_pdf,
    standard_normal_cdf,
    normal_pdf,
    normal_cdf,
)


def model(p):
    x = p[0]
    return [
        standard_normal_pdf(x),
        standard_normal_cdf(x),
        normal_pdf(x, mean=2.0, std=1.5),
        normal_cdf(x, mean=2.0, std=1.5),
    ]


result = autoia(model, [Interval(-0.5, 0.5)])
print(result)
```

Common SciPy forms such as `scipy.special.erf`, `scipy.special.ndtr`, and supported `scipy.stats.norm` PDF/CDF syntax are locally redirected when validated values are present.

---

13. Bracketed implicit scalar roots

Engineering states are often defined by

$$F(y, p) = 0.$$

AutoIA supports bracketed `scipy.optimize.root_scalar` as a semantic primitive.

```python
import numpy as np
from scipy.optimize import root_scalar
from autoia import autoia, Interval


def colebrook_resid(f, Re, rel_roughness):
    if f <= 0:
        return np.nan
    term = rel_roughness/3.7 + 2.51/(Re*np.sqrt(f))
    return 1.0/np.sqrt(f) + 2.0*np.log10(term)


def model(p):
    Re, rr = p
    return root_scalar(
        colebrook_resid,
        bracket=[0.0008, 0.1],
        args=(Re, rr),
    ).root


result = autoia(
    model,
    [
        Interval(90000.0, 110000.0),
        Interval(0.0008, 0.0012),
    ],
)
print(result)
```

SciPy proposes a nominal root. AutoIA independently proves a uniform bracket sign change, uniqueness through a nonzero residual derivative, and a local predictor-plus-correction enclosure. The requested SciPy bracketed method affects proposal behavior, not the proof standard.

---

14. Square nonlinear systems with `fsolve`

```python
import numpy as np
from scipy.optimize import fsolve
from autoia import autoia, Interval


def residual(y, a, b):
    return np.array([
        y[0] + 0.2*y[1] - a,
```

```
        0.1*y[0] + y[1] + 0.05*y[0]*y[1] - b,
    ])


def model(p):
    return fsolve(residual, [1.0, 1.5], args=(p[0], p[1]))


result = autoia(
    model,
    [Interval(0.9, 1.1), Interval(1.4, 1.6)],
)
print(result)
```

The V1 theorem proves a unique solution inside a local tube around the dependency-bearing implicit predictor. It does **not** claim that no other remote solution exists elsewhere.

Near singular Jacobians, bifurcations, or merged branches, certification can fail. That is correct fail-closed behavior.

---

15. Cross-module ordinary engineering helpers

AutoIA can locally and lazily prepare source-backed imported Python helpers when semantic values reach them. The helper module is not modified.

`engineering.py`:

```
from scipy.optimize import root_scalar


def residual(y, a):
    return y*y - a


def solve_state(a):
    return root_scalar(residual, bracket=[0.5, 2.0], args=(a,)).root
```

`main.py`:

```
import engineering
from autoia import autoia, Interval


def model(p):
    return engineering.solve_state(p[0])
```

```
print(autoia(model, [Interval(1.0, 1.21)]))
```

"Locally" means AutoIA creates a private prepared execution clone for the current proof. "Lazily" means it does so only when the helper is actually reached with AutoIA semantic values. Numeric-only calls continue to use the original helper.

---

16. Fixed-data `RegularGridInterpolator`

AutoIA V1 supports fixed numeric rectilinear grids and fixed numeric table values for

```
scipy.interpolate.RegularGridInterpolator(..., method="linear")
```

with uncertainty in the query coordinates.

```
import numpy as np
from scipy.interpolate import RegularGridInterpolator
from autoia import autoia, Interval

X = np.array([0.0, 0.5, 1.0, 1.5])
Y = np.array([-1.0, 0.0, 1.0])
VALUES = np.empty((len(X), len(Y)))
for i, x in enumerate(X):
    for j, y in enumerate(Y):
        VALUES[i, j] = 2.0 + 0.5*x - y + 3.0*x*y

TABLE = RegularGridInterpolator((X, Y), VALUES, method="linear")


def model(p):
    return TABLE((p[0], p[1]))


result = autoia(
    model,
    [Interval(0.2, 0.9), Interval(-0.4, 0.6)],
)
print(result)
print("cell/path branches =", len(result.branch_results))
```

The input box crosses grid boundaries. AutoIA treats those boundaries as path decisions. Within each selected cell it evaluates the exact multilinear interpolation formula in the original dependency-bearing query coordinates.

Supported interpolation details

V1 supports:

- ascending or descending rectilinear axes;
- scalar and fixed vector-valued table data;
- query boxes inside one cell or crossing multiple cells;
- `bounds_error=True` when the whole certified domain remains inside the grid;
- `bounds_error=False` with a finite constant `fill_value`;
- `bounds_error=False, fill_value=None` for linear extrapolation.

V1 deliberately does not claim:

- `nearest`, `slinear`, `cubic`, `quintic`, or `pchip` validated semantics;
- uncertain grid coordinates;
- uncertain table values.

---

17. Recognized unsupported operations

A mature validation tool should distinguish "not supported" from "proof failed."

For example, V1 recognizes `scipy.linalg.eigh` with uncertain matrix entries but does not yet provide a rigorous eigenvalue/eigenvector theorem:

```
import numpy as np
from scipy.linalg import eigh
from autoia import autoia, Interval, AutoIAError


def modal_model(p):
    k = p[0]
    K = np.array([[2.0*k, -k], [-k, 2.0*k]])
    M = np.array([[2.0, 0.0], [0.0, 1.0]])
    w = eigh(K, M, eigvals_only=True)
    return np.sqrt(w[0])/(2.0*np.pi)


try:
    autoia(modal_model, [Interval(90.0, 110.0)])
except AutoIAError as exc:
    print(type(exc).__name__)
    print(exc)
```

The expected category is `UnsupportedOperationError`, with a message that `scipy.linalg.eigh` is recognized but no certificate was produced.

This is better than allowing SciPy to fail internally with an object-array error, and much better than returning an uncertified nominal eigenvalue.

---

18. Typed failure semantics

Catch the base class when you only need "no certificate," or a specific subclass when behavior depends on why certification stopped.

```
from autoia import (
    AutoIAError,
    InvalidInputError,
    UnsupportedOperationError,
    CertificationError,
    ResourceLimitError,
    NumericalLimitError,
    ModelExecutionError,
    InternalAutoIAError,
)
```

Meanings:

- `InvalidInputError`: malformed AutoIA call/configuration;
- `UnsupportedOperationError`: program semantics outside the supported language;
- `CertificationError`: meaningful request, but a required proof could not be established;
- `ResourceLimitError`: a hard branch/refinement/resource budget stopped the proof;
- `NumericalLimitError`: trusted numerical arithmetic encountered a limit;
- `ModelExecutionError`: the user's model itself raised an exception;
- `InternalAutoIAError`: an AutoIA invariant failed and should be treated as a software bug.

Do not catch `InternalAutoIAError` and continue as though the calculation were merely conservative. It is a defect signal.

---

19. Symmetric positive-definite matrix families

AutoIA V1 can certify common calculations built from a parameter-dependent symmetric positive-definite (SPD) matrix family. If

$$A(p) = A(p)^T \succ 0$$

for every physical input p in the supplied box, AutoIA can certify the matrix family and then evaluate quantities such as

$$\log\det A(p)$$

and

$$q(p) = b(p)^T A(p)^{-1} b(p).$$

Here $b(p)$ may depend on the same uncertain physical inputs as the matrix. The important point is that AutoIA does not first replace every matrix entry by an independent scalar interval. Shared input dependence is retained through a constant preconditioner and a validated matrix-series theorem.

A copy-paste example is:

```
import numpy as np
from autoia import autoia, Interval, quad_solve_spd


def model(p):
    x, y = p
    off = 0.10*x - 0.04*y
    A = np.array([
        [2.0 + 0.15*x*x + 0.05*y, off],
        [off, 1.5 + 0.08*y*y - 0.03*x],
    ], dtype=object)
    b = np.array([1.0 + 0.2*x, 0.5 - 0.1*y], dtype=object)

    sign, logdet = np.linalg.slogdet(A)
    det = np.linalg.det(A)
    q = quad_solve_spd(A, b)
    return [sign, logdet, det, q]


result = autoia(
    model,
    [Interval(-0.25, 0.25), Interval(-0.20, 0.30)],
)
print(result)
```

For a proved SPD family, `np.linalg.slogdet(A)` returns an exact sign of `+1` and a validated log determinant. `np.linalg.det(A)` uses the validated SPD log determinant followed by a validated exponential. `quad_solve_spd(A, b)` returns a rigorous enclosure of $b^T A^{-1} b$.

The floating-point Cholesky factor used internally is only a proposal/preconditioner. It is not trusted as a proof. AutoIA separately verifies symmetry, positive

eigenvalue bounds, the matrix-log/inverse-series convergence conditions, and rigorous remainder bounds.

If a broad box is difficult, smaller physical boxes normally sharpen these quantities significantly. See `22_spd_matrix_families.py`.

---

20. Prepared sessions and repeated boxes

If the same model will be certified many times, use `prepare`:

```
import numpy as np
from autoia import prepare, Interval


def model(p):
    x, y = p
    return np.array([np.exp(x) + y*y, x*y])


session = prepare(model, cache_max_entries=32)

parent = [Interval(-1.0, 1.0), Interval(0.0, 2.0)]
child = [Interval(-0.25, 0.25), Interval(0.5, 1.0)]

r1 = session.certify(parent)
r2 = session.certify(child)
r3 = session.certify(child)

print(r1.cache_info)
print(r2.cache_info)
print(r3.cache_info)
print(session.stats)
```

A repeated identical box can use the exact certificate cache. A contained child box can sometimes use rigorous restriction/recentering of a cached parent certificate without re-executing the engineering model.

Many-box use

```
boxes = [
    [Interval(-0.5, -0.4), Interval(0.9, 1.0)],
    [Interval(0.0, 0.1), Interval(1.0, 1.1)],
    [Interval(0.4, 0.5), Interval(1.1, 1.2)],
]

lo, hi, info = session.enclose_many_bounds(
```

```
    boxes, workers="auto", parallel="auto", return_info=True
)
print(lo)
print(hi)
print(info)
```

Prepared sessions are important for optimization, design sweeps, branch-and-bound, reliability analysis, and repeated uncertainty studies. The many-box interface only evaluates boxes supplied by the caller; it is not itself a global optimizer.

---

21. Parallel CPU proof and optional MPS/CUDA proposal acceleration

Repeated independent boxes can be certified on multiple CPU cores without changing the proof. Use a prepared session:

```
from autoia import prepare, Interval


def model(p):
    x, y = p
    return x*x + y


boxes = [
    [Interval(-1.0, -0.5), Interval(0.0, 0.2)],
    [Interval(-0.5, 0.0), Interval(0.0, 0.2)],
    [Interval(0.0, 0.5), Interval(0.0, 0.2)],
    [Interval(0.5, 1.0), Interval(0.0, 0.2)],
]

session = prepare(model)
results, info = session.enclose_many(
    boxes,
    workers="auto",
    parallel="auto",
    return_info=True,
)

print(info)
```

The V1 high-level default is **safe auto scheduling**. With `workers="auto"` and `parallel="auto"`, AutoIA first groups the finite workload using `chunk_size` (64 by default). Fewer than four chunks stay serial so a small job does not pay process-pool startup cost. At four or more chunks, AutoIA uses at most

one worker per chunk, capped by the available CPU count; it prefers process workers when the model can be serialized and otherwise uses threads. Explicit `workers=` or `parallel=` values override this scheduling policy. The four-chunk crossover is a deterministic performance heuristic, not part of the proof.

When optional `cloudpickle` is installed, many local functions and closures can participate in process execution. Each process owns an independently prepared trusted CPU backend. The returned `ManyBoxExecutionInfo` is diagnostic only. It records the requested and resolved worker count, process/thread/serial mode, chunk size, elapsed time, serializer, and a `scheduling_note` explaining the auto choice. None of these fields is consulted to decide whether a mathematical enclosure is valid.

Apple Metal and NVIDIA CUDA

AutoIA V1 also exposes optional Torch-backed proposal helpers:

```
from autoia import available_accelerators, proposal_map

print(available_accelerators())
```

On Apple Silicon, PyTorch's `mps` device can be used for batched proposal calculations. On NVIDIA hardware, `cuda` can be requested. These GPU calculations are **not proofs**. In V1, `device="auto"` deliberately selects CPU. AutoIA does not attempt to guess whether an arbitrary Torch proposal is compute-heavy enough for GPU dispatch to pay off; request `device="mps"` or `device="cuda"` explicitly when desired. Their result object has

```
proposal.machine_rigorous == False
```

and final certificates still come from independent CPU AutoIA evaluation. This is the V1 trust boundary: GPU for proposal/assist throughput, multicore CPU for trusted verification.

Run `examples/check_apple_mps.py` on an Apple Silicon machine and `examples/check_cuda_accelerator.py` on a CUDA machine. The numbered `23_parallel_and_accelerators.py` example runs everywhere and demonstrates an explicit accelerator opt-in when one is available. Example 24 then demonstrates the one-shot, compact-bound, and streaming many-box APIs, while Example 25 applies the SPD machinery to a 3-by-3 RBF-like matrix family and shows certified shrinking-box convergence.

---

22. Application-specific theorems without contaminating the core

When an application has mathematics stronger than generic interval analysis, use the public application theorem interface instead of adding application knowl-

edge to AutoIA itself.

Monotonicity example

```
import numpy as np
from autoia.application import ApplicationBox, application_theorem


@application_theorem(name="exp is increasing", target="f")
def exp_upper(box, cert):
    x_upper = cert.point(box.upper[0])
    return cert.exp(x_upper)


box = ApplicationBox(np.array([-0.7]), np.array([1.2]))
bound = exp_upper(box)
print(bound.value)
```

Portfolio of independent upper bounds

```
from autoia.application import minimum_upper_bound

best = minimum_upper_bound(bound1, bound2, bound3)
```

If all supplied bounds are rigorous upper bounds for the same target, their minimum is still a rigorous upper bound. The analogous operation for lower bounds is `maximum_lower_bound`.

This is the intended extension path for applications such as Bayesian optimization: application packages depend on AutoIA, not the reverse.

---

23. How to read a result

The most important fields are:

```
result.lower
result.upper
result.width
result.machine_rigorous
result.output_shape
```

For automatic configuration:

```
result.automatic_configuration
result.configuration_summary
result.configuration_diagnostics
result.explain_configuration()
```

For path-aware results:

```
result.branch_results
result.branch_boxes
result.branch_decisions
result.branch_traces
result.path_events
```

For semantic primitives:

```
result.certificates
```

A certificate dictionary is diagnostic evidence describing the theorem that was verified. The public numeric guarantee remains the returned output interval.

---

24. Sampling is useful, but never the proof

For an engineering validation study, it is good practice to compare the certified interval with a dense sample:

```
import numpy as np

xs = np.linspace(lower, upper, 10001)
vals = np.array([ordinary_model(float(x)) for x in xs])

assert result.lower[0] <= vals.min()
assert vals.max() <= result.upper[0]
```

This can catch bugs during development and gives a sense of sharpness. It does not establish the universal statement. A million samples can still miss an unsampled extremum.

A second useful diagnostic is a shrinking-box study. If the physical box contracts around a regular point, a healthy enclosure method should generally shrink toward the point value.

---

25. Recommended workflow for new engineering code

1. Run the ordinary model numerically first and establish its physical domain.
2. Wrap only the uncertain physical inputs in an AutoIA input vector. Do not rewrite the model in interval syntax.
3. Start with `autoia(model, box)` and no expert representation/order controls.
4. If it certifies, compare against diagnostic sampling and perform at least one shrinking-box check.

5. Inspect `result.explain_configuration()` and `result.path_events` only if you need to understand cost or looseness.
6. If AutoIA fails, classify the failure before changing anything:
   - unsupported semantic;
   - proof could not be established;
   - resource limit;
   - user-model exception;
   - internal bug.
7. Avoid application-specific workarounds in the core. If there is a general missing semantic, add a general theorem-backed primitive. If there is an application theorem, put it in the application layer.
8. Freeze successful outside-code cases as regressions.

---

26. The complete numbered example suite

The canonical sequence is:

- `00_smoke_test.py`
- `01_scalar_nonlinear.py`
- `02_vector_numpy.py`
- `03_linear_solve.py`
- `04_branching.py`
- `05_subdivision.py`
- `06_prepared_many_box.py`
- `07_generator_cancellation.py`
- `08_normal_functions.py`
- `09_failure_handling.py`
- `10_ei_like.py`
- `11_application_monotonicity.py`
- `12_application_compose.py`
- `13_application_lipschitz.py`
- `14_application_portfolio.py`
- `15_implicit_solves.py`
- `16_cross_module_implicit_solves.py`
- `17_regular_grid_interpolator.py`
- `18_recognized_unsupported_eigh.py`
- `19_expert_representations_and_ranging.py`
- `20_domain_refinement_and_closed_boundaries.py`
- `21_cross_module_interpolation.py`
- `22_spd_matrix_families.py`
- `23_parallel_and_accelerators.py`
- `24_many_box_workflows.py`
- `25_rbf_like_spd_family.py`

Run all of them in fresh subprocesses with:

```
python examples/run_all_examples.py
```

The feature-by-feature mapping is in `examples/COVERAGE_MATRIX.md`.

---

27. Final mental model

Think of AutoIA as maintaining the invariant

$$\text{exact engineering value} \in \text{dependency-bearing validated representation}.$$

It tries to retain original physical dependence as long as practical, uses shared generators when approximation-error correlation matters, treats important numerical operations by their mathematical semantics, follows uncertain execution paths rather than erasing them, subdivides only as a proof/sharpness tool, and confines the final correctness argument to a small trusted numerical layer.

A useful summary is:

```
ordinary engineering Python
        |
        v
private local semantic dispatch
        |
        v
physical-input polynomial / interval / shared-generator values
        |
        +--> certified elementary functions
        +--> certified linear solve
        +--> certified root_scalar / fsolve
        +--> certified linear RegularGridInterpolator
        +--> certified SPD matrix-family logdet / inverse quadratic forms
        +--> path-aware branches and domain refinement
        +--> multicore CPU many-box proof
        +--> optional non-rigorous MPS/CUDA proposal assist
        |
        v
rigorous range methods
        |
        v
outward-rounded binary64 output intervals
```

The controller is allowed to be heuristic because every accepted branch, semantic primitive, and final bound must still pass an independent rigorous proof obligation.